despite being based on different inputs. If we had normalized these inputs to be within [–1,1], for example, `tanh` might return 0.5 for one input and 0.6 for the other, a meaningful difference.

So rather than just giving the sum of the individual action vectors to our Q function, we will give it the sum divided by the total value of all the elements, which will normalize the elements in the resulting vector to be between [0,1]. For example, we will compute [1,0] + [0,1] = [1,1]/2 = [0.5,0.5]. This normalized vector will sum to 1, and each element will be between [0,1], so what does that remind you of? A probability distribution. We will, in essence, compute a probability distribution over the actions of the nearest neighbors, and give that vector to our Q function.

Computing the mean field action vector

In general, we compute the mean field action vector with this formula,

$$a_{-j} = \frac{1}{N} \sum_{i=0}^{N} a_i$$

where a_{-j} is just a notation for the mean field of the neighboring agents around agent j, and a_i refers to the action vector for agent i, which is one of agent j's neighbors. So we sum all the action vectors in the neighborhood of size N for agent j, and then we divide by the size of the neighborhood to normalize the results. If the math doesn't suit you, you will see how this works in Python soon.

This approach is called a *mean field approximation*, or in our case, *mean field Q-learning* (MF-Q). The idea is that we compute a kind of average magnetic field around each electron rather than supplying the individual magnetic fields of each neighbor (figure 9.19). The great thing about this approach is that the mean field vector is only as long as an individual action vector, no matter how big our neighborhood size is or how many total agents we have.